Robust Time-Series Forecasting Under Distribution Shift: A Transformer with Online Conformal Calibration

Sahil Chaudhari, Sakshaat Raut and Ritika Karande

1. Background

Companies such as Microsoft, Amazon and Google rent out computers over the internet. Each rented computer is called a virtual machine, or VM. To run this service well, a provider must know how busy each VM will be in the near future, so it can keep the right number of machines switched on.

This is a time-series forecasting task. We look at a VM's past CPU usage (0% to 100%, recorded every 5 minutes) and predict the next hour.

Transformers are the type of neural network behind tools like ChatGPT. They are very good at finding patterns in sequences, including time series. We use a well-known forecasting transformer called PatchTST (Nie et al., 2023).

A good forecast gives a range, not a single number. For example: "CPU usage will be between 30% and 50%, and we are 90% sure."

2. Significance

Cloud providers plan around the top of the forecast range. If the top is too low, VMs get overloaded and customers see slow or failing applications. If it is too high, the provider pays for machines that sit idle.

The difficulty is that VMs do not behave the same way forever: a software release or a rush of users can change a VM overnight.

After such a change, a forecasting model still says "90% sure", but it is no longer right 90% of the time, and nothing warns the operator. Fixing this makes transformer forecasts safe to rely on in real systems.

3. Statement of the problem

A transformer's forecast range is only reliable while a VM behaves as it did during training. Our goal: given the last 24 hours of a VM's CPU usage, predict the next hour as a range that contains the true value about 90% of the time, both before and after a sudden change.

We will do this in two steps and compare them:

1. **Baseline.** Train the PatchTST transformer to predict ranges for VM CPU usage.
2. **Our improvement.** Add a small correction layer on top of the same model. It checks every new forecast against what really happened. When the model misses, it widens the next ranges; when the model keeps being right, it narrows them. This technique is called online conformal prediction (Gibbs and Candès, 2021; Angelopoulos et al., 2023). The transformer itself is not retrained.

We expect the baseline to lose reliability after a sudden change, and the improved version to recover within a few hours with only slightly wider ranges.

4. How to obtain the training data

We use the Azure Public Dataset (2019 VM trace), released by Microsoft (Cortez et al., 2017). It is free, public and needs no registration. It records the CPU usage of about 2.7 million real VMs every 5 minutes for 30 days.

The full dataset is very large (156 GB), so we will work with a representative sample, prepared in three steps:

- **Selection.** We will randomly select 2,000 VMs that run for the full month and are not idle.
- **Cleaning.** We will fill short gaps of up to 30 minutes and exclude VMs with too much missing data.
- **Splitting.** We will divide the month by time: days 1-18 for training, days 19-21 for tuning, and days 22-30 for the final test.

5. Metrics for validation and verification

Metric	What it tells us	Goal
Coverage	How often the true value falls inside the predicted range	Close to 90%
Range width	How wide the range is, in CPU percentage points	As narrow as possible
Recovery time	How long coverage takes to return to 90% after a sudden change	A few hours
MAE and RMSE	How far the middle of the forecast is from the true value	Better than simple guesses

We will plot coverage and width over time, with the moment of change marked. We will test on VMs that change naturally during the test days, and on changes we add ourselves at a known moment.

All results will come from the test days, which the models never see during training. To verify the code, we will first run it on made-up data where we know the right answer. We will also compare the transformer against two simple guessing rules: "same as the last reading" and "same as this time yesterday".

6. Deep learning framework

We implement the project in PyTorch and train the full model on the University of Maryland's Zaratan cluster. As the assignment allows, we follow existing open-source implementations, cite them, and adapt them to our problem:

- **Transformer:** the official PatchTST code (Nie et al., 2023), changed to predict a range instead of a single number.
- **Correction layer:** the conformal-time-series code (Angelopoulos et al., 2023). We connect it to our transformer's forecasts, one VM at a time.

References

1. Angelopoulos, A. N., Candès, E. J., and Tibshirani, R. J. Conformal PID Control for Time Series Prediction. NeurIPS, 2023.
2. Cortez, E., et al. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. SOSP, 2017.
3. Gibbs, I., and Candès, E. J. Adaptive Conformal Inference Under Distribution Shift. NeurIPS, 2021.
4. Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR, 2023.